VikSchema

Designing a Responsibility Driven Living System
for Scalable Product Architecture

What is a Structural Debt

? The Problem

Structural Debt occurs when the underlying architecture of a design system is not defined early, forcing teams to patch inconsistencies instead of building on a stable foundation.

69

These are not isolated UI problems.
They are symptoms of structural debt.

Overtime this creates:

- Duplicate Components
- Variant Explosion
- Inconsistent Spacing and Styling
- Unclear Ownership and Dependencies
- Systems that Become Harder to Scale and Maintain

Why Systems Become Fragile

? The Hidden Problem

Most design systems attempt to standardize components.

But as Systems grow, structure, styling, layout, and context become mixed together.

The problem is not simply inconsistency.

The problem is that systems are often designed around composition, but not around responsibility.

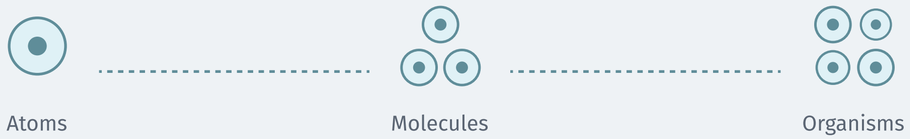
This creates:

- Ambiguous Responsibilities
- Tightly Coupled Components
- Unpredictable Behavior Across Contexts
- Increasing System Complexity Over Time

Why Existing Models Break

? Composition Alone Is Not Enough

Atomic Design introduced an important way to think about UI structure:



It helped teams organize components by composition.

Atomic Design organizes by complexity.

VikSchema organizes by responsibility.

But as systems scale, new problems emerge:

Styling Decisions Leak Across Layers

Visual decisions are not contained, leading to inconsistent components.

Components Become Context-Dependent

Components behave differently across screens, layouts, or environments.

Responsibilities Overlap

Unclear boundaries create confusion about where decisions belong.

Scaling Across Products Becomes Difficult

The model doesn't support multi-brand, multi-product ecosystems.

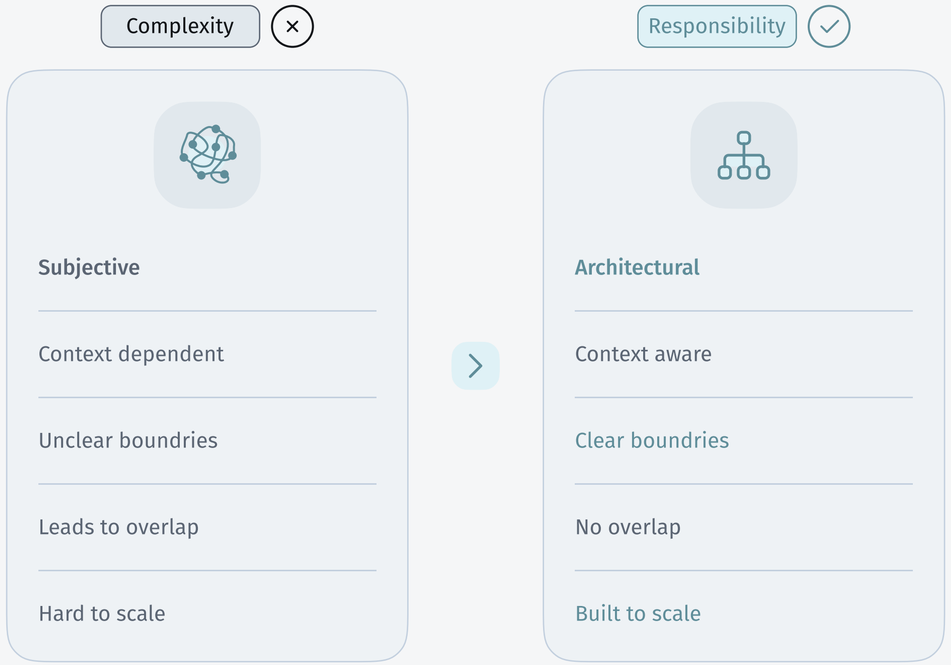
Responsibility Over Complexity

Complexity is subjective.
Responsibility is architectural.

Instead of asking:
How complex is this component?

VikSchema asks:
What responsibility does this layer hold?

Complexity becomes the outcome of responsibilities combining together.
Not the foundation of the system itself.



This creates:

- Clear System Boundries
- Predictable dependency Flow
- Scalable Composition
- Controlled System Growth

The Living System Architecture

VikSchema structures UI as a living system.

Gene	Cell	Tissue	Organ	Organism	Colony
Foundational Information	Structure and Containment	Functions and Behavior	Capabilities and Intractions	Context and System	Coordinated Ecosystem

Each layer has:

01 • Clearly Defined Responsibility

02 • Strict Dependency Boundries

03 • A Predictable Role Within the System

Lower layers cannot depend on higher layers.

This ensures that complexity grows in a controlled and scalable way.

Why a Living System

Why Biology

Biological systems are naturally scalable systems.

They organize complexity through:

- Specialization
- Clear Responsibilities
- Dependency Boundaries
- Hierarchical Composition
- Controlled Growth

A gene does not behave like a tissue.

A tissue does not behave like an organ.

Each layer exists because it serves a different responsibility within the system.

The biological model was chosen not as a metaphor for creativity, but as a framework for scalable system organization.

Responsibility Mapping

Foundational Layers

Gene

Defines foundational information and primitives.

Biological Role:

Encodes foundational instructions.

UI Responsibility:

Raw primitives and tokens.

Example:

Icon, Typography

Cell

Defines basic structure and containment.

Biological Role:

Basic structural unit.

UI Responsibility:

Structural composition primitives.

Example:

Label

Tissue

Introduces interaction, behavior, and coordinated function.

Biological Role:

Cells working together toward a shared function.

UI Responsibility:

Interactive and styled components. Introduces styles but does not apply them yet.

Example:

Input, Checkbox, Button

Responsibility Mapping

Compositional Layers

Organ

Combines multiple tissues into functional modules.

Biological Role:

Specialized functional systems.

UI Responsibility:

Feature-level composition.

Example:

Form Field.

Organism

Defines context and full-screen behavior.

Biological Role:

Context-aware system coordination.

UI Responsibility:

Screen-level composition and hierarchy resolution.

Example:

Search Flow, Login Flow, Checkout Flow

Colony

Represents multiple systems working together.

Biological Role:

Coordinated ecosystem behavior.

UI Responsibility:

Full product architecture.

Example:

Web app, Mobile app, Admin platform, Multi-product ecosystem

Why This Structure Scales

Growth Through Responsibility

Traditional systems often scale by adding more components.

VikSchema scales by combining responsibilities across layers.

Lower layers remain stable.

Higher layers adapt to context.

This allows the system to evolve without structural fragmentation.

This creates:

- Predictable Growth
- Clearer Ownership
- Reusable Structures
- Controlled Complexity
- Scalable Composition

Designed for Systems

Not Just Screens

VikSchema was designed to support scalability across:

- Design
- Engineering
- Product Architecture
- Multi-Product Ecosystems

For Designers

- Reduced Cognitive Load
- Predictable Composition
- clearer system boundaries

For Developers

- Cleaner Design-to-code Mapping
- Separation of Concerns
- Reusable Architecture

Product/ Organization

- Scalable Foundations
- Faster Feature Development
- Consistent Product Experiences Across Platforms

Shared Outcome:
Reduced complexity, increased predictability, faster scaling

Design to Code

A shared Design-to-Code Architecture

VikSchema aligns design structure with engineering architecture.

Each layer maps to a predictable level of implementation:

This creates a shared system language between design and engineering.

The system becomes easier to scale, maintain, and reason about across teams.

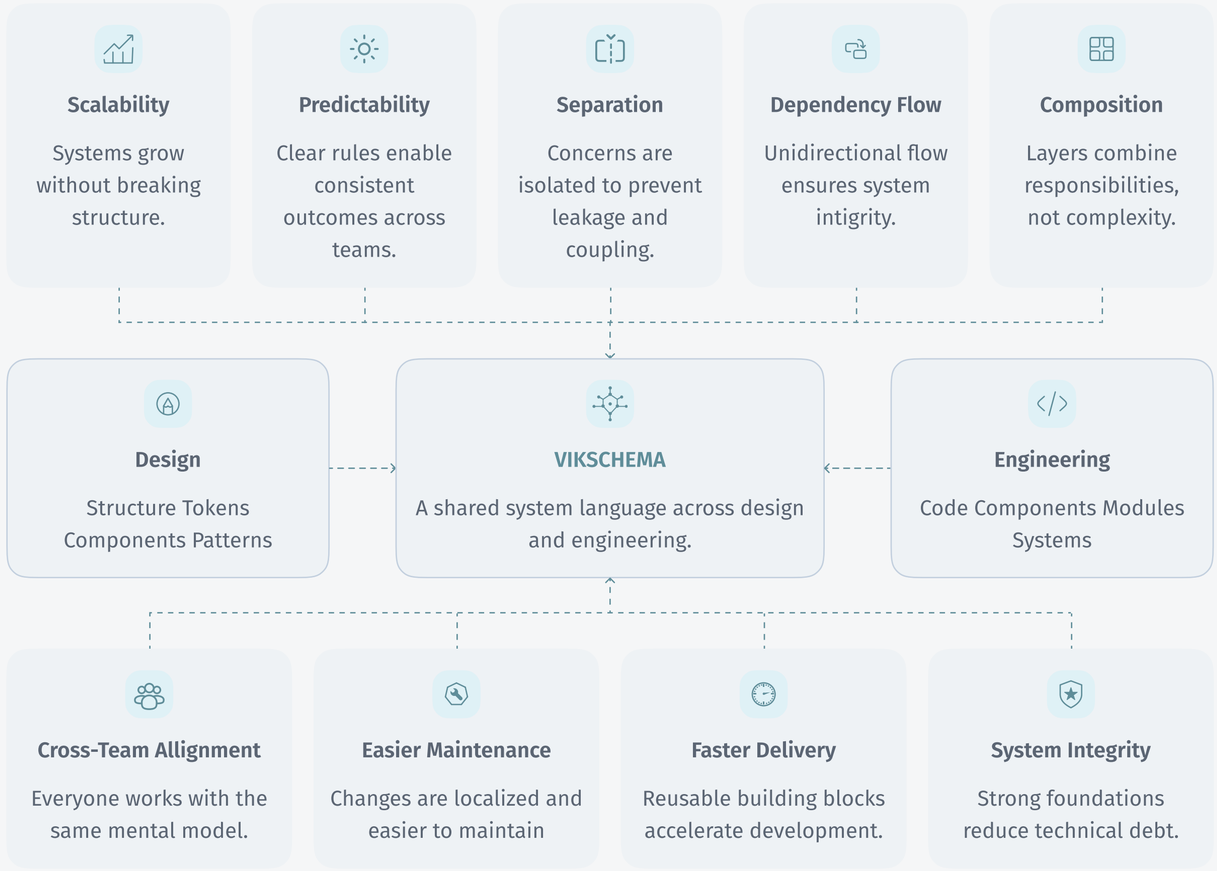
Gene	Tokens / Constants
Cell	Structural Layout Primitives
Tissue	Reusable Interactive Components
Organ	Feature-Level Modules
Organism	Screen-Level Composition
Colony	Product Architecture

Core System Decisions

Key decisions include:

- Multi-File Architecture
- Progressive Token Layering
- Separation of Tokens, Components, and Product
- Responsibility-Based Component Layers
- Viewport and Layout Separation
- Density and Rhythm Separation
- Relative Elevation Systems
- Slot-Based Composition
- Constraint-Aware Figma Architecture
- Context aware styling

OUR ARCHITECTURAL DECISIONS CREATE:



Each decision addresses a specific scaling problem commonly found in large design systems.

Multi-File Architecture

Traditional design systems often grow inside a single file or loosely structured collection of files.

As systems scale, dependencies become unclear and system boundaries begin to blur.

VikSchema introduces a layered file architecture:



Each file represents a stage in system construction, not just storage.

This creates:

- Clear Ownership Boundaries
- Predictable Dependency Flow
- Scalable Multi-Product Support
- Reduced System Fragility

Progressive Token System

Many token systems become difficult to scale because values, themes, and usage are mixed together.

VikSchema structures tokens progressively:

Assets → Brand → Core → Definition & Experience

Where:

- Assets Define Raw Values
- Brand Defines Identity and Themes
- Core Defines Abstraction
- Definition & Experience exposes usable tokens

Token layer purpose

Assets	Define raw values.
Brand	Define identity and themes.
Core	Define abstraction
Definition & Experience	Exposes usable tokens

This ensures:

- Predictable Token Flow
- Traceable Dependencies
- Scalable Theming
- Reduced Token Complexity

Only the final layer is exposed to the UI.

Separation of Tokens, Components, and Product

Separation of System Layers

In many systems, tokens, components, and product-specific UI become tightly coupled.

This reduces reuse and increases maintenance complexity.

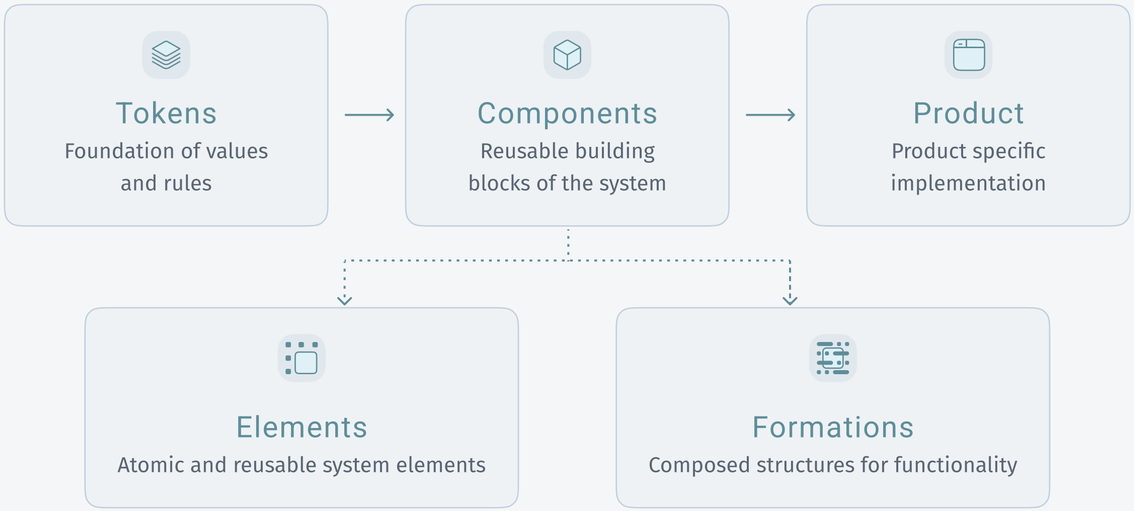
VikSchema separates the system into:

Tokens → Components → Product

Components are further divided into:

Elements → Formations

System Separation



This separation allows:

- Reusable System Foundations
- Independent Product Evolution
- Scalable Multi-Product Ecosystems
- Clearer Ownership and Responsibilities

Component Layer System

Internal Component Architecture

As components grow, layout and styling decisions often become tightly coupled.

Small visual changes can unintentionally affect structure and behavior.

VikSchema introduces a strict internal component structure:

Wrapper → Layout → Style → Content

This separates: Composition, Spacing, Styling, and Content handling

Internal Component Architecture

- **Wrapper** — Define the outer boundary and component contract.
- **Layout** — Manages internal structure and spatial arrangement.
- **Style** — Handles visual treatment and appearance.
- **Content** — Manages dynamic content and data rendering.

The result is:

- Predictable Components
- Easier Maintenance
- Safer Iteration
- Scalable Component Architecture

Viewport vs Layout Separation

Environment vs Composition

In many systems, screen constraints and layout logic become mixed together.

This creates inconsistent spacing and device-specific complexity.

VikSchema separates:

Viewport → Environment Constraints
Layout → Component Composition

Viewport Handles

- Margins
- Safe Area
- Screen Rhythm
- Device Constraints

Layout Handles

- Stacking
- Arrangement
- Structural Composition

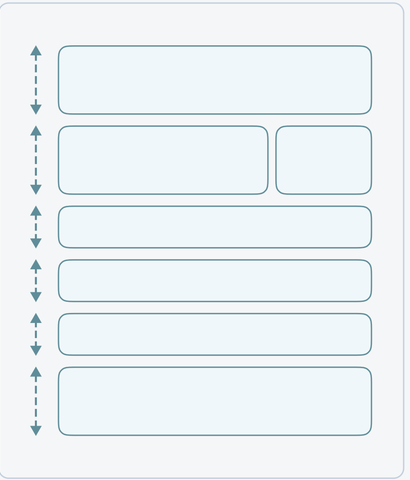
System Separation

Viewport
Environment



Define boundaries and environment constraints for the screen.

Layout
Composition



Defines how components are arranged and composed within the boundaries.

This separation improves scalability across devices and platforms.

Density vs Rhythm

Internal Spacing vs External Rhythm

Not all spacing solves the same problem.

Using a single spacing system for everything creates inconsistent layouts and unpredictable component behavior.

VikSchema separates spacing into:

Density → Internal Component Spacing
Rhythm → External Layout Spacing

Density controls

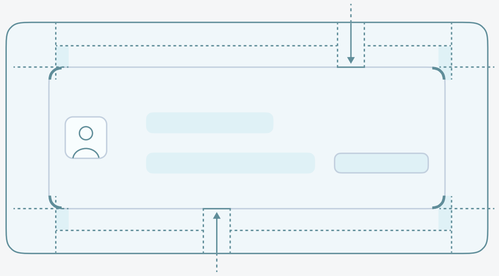
- Padding
- Internal Gaps
- Component Compactness

Rhythm controls

- Section Spacing
- Screen Flow
- Visual Pacing

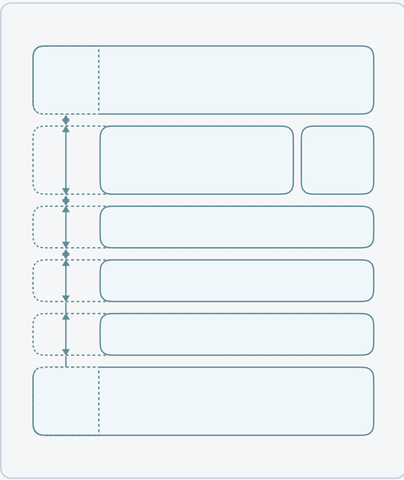
Spacing Difference

Density
Internal Component Spacing



Manages spacing within components
to control compactness and readability.

Rhythm
External Layout Spacing



Manages spacing between sections
to create flow and visual pacing.

This creates more predictable layouts and balanced interfaces.

Elevation as a Relative System

Relative Elevation Architecture

Traditional elevation systems often assign fixed hierarchy values to components.

This breaks when components are reused across different contexts.

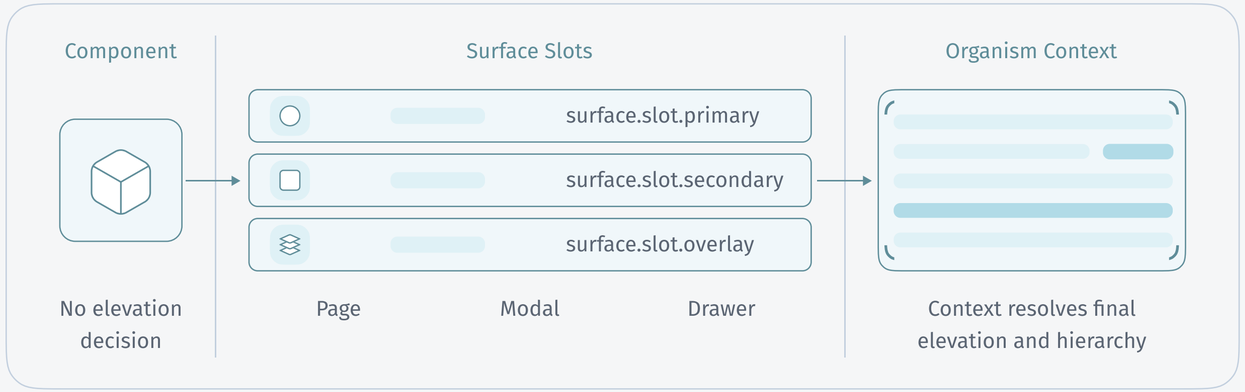
VikSchema introduces relative elevation using surface slots:

- surface.slot.primary
- surface.slot.secondary
- surface.slot.overlay

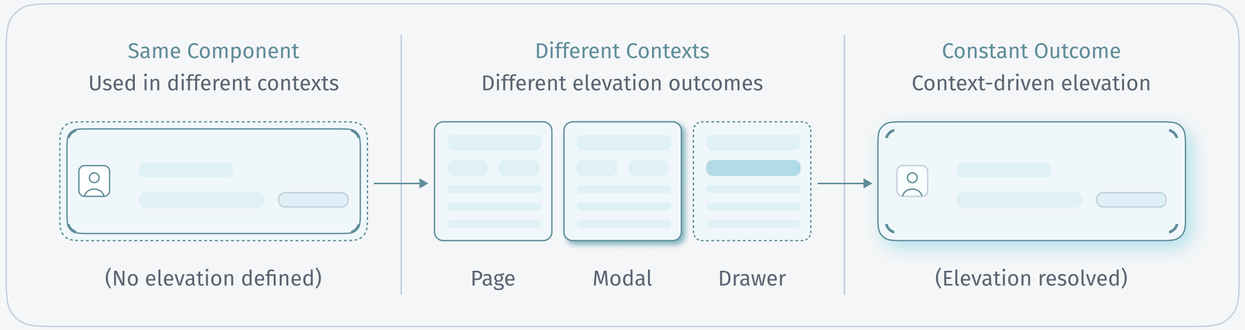
This creates

- Reusable components
- Consistent Hierarchy
- Scalable contextual behavior

Elevation Architecture



Example Usage



Components do not define their final elevation.
Context resolves elevation at the Organism level.

Slot-Based Composition

Structure Over Variants

As components evolve, systems often accumulate excessive variants to support small structural differences.

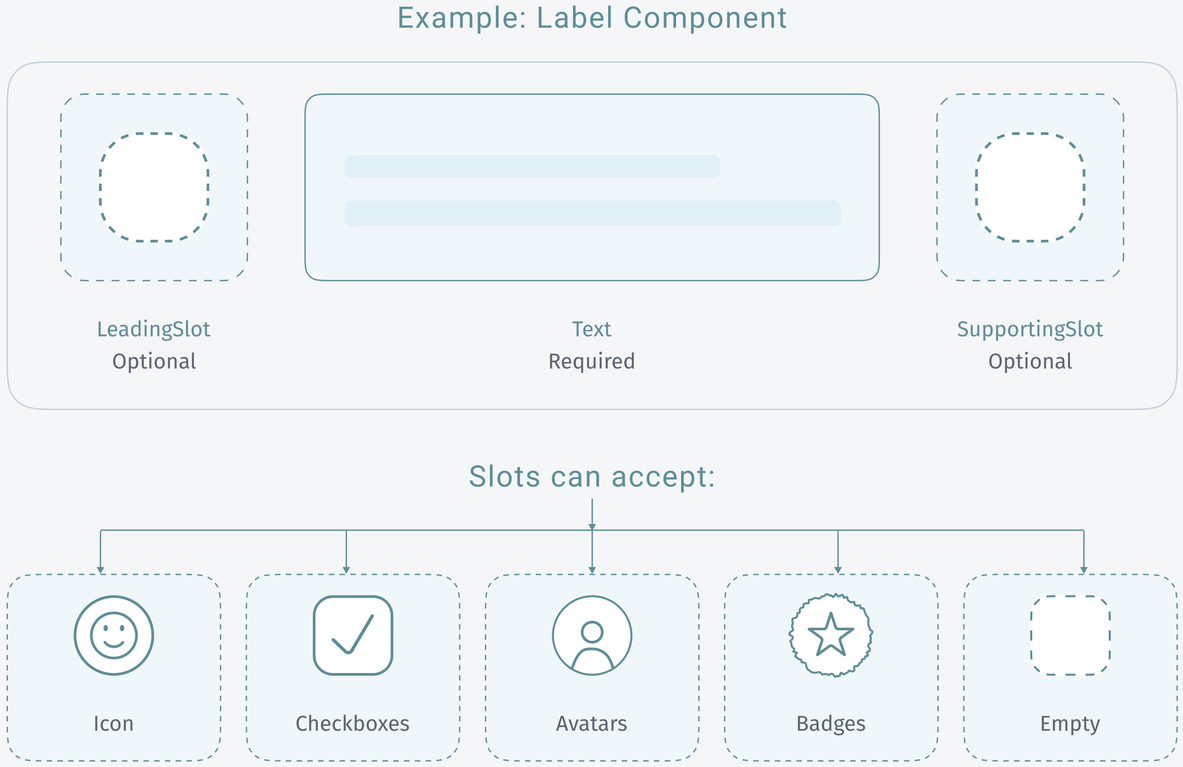
This leads to variant explosion.

VikSchema uses a slot-based composition model.

Components define structure, not fixed outcomes.

This reduces variants while preserving flexibility and scalability.

Solt-Based Composition



This model enables greater reuse, cleaner structure, and scalable component architecture.

Figma Constraint-Driven Architecture

Designed Around Real Tool Constraints

Many design systems are built around ideal workflows rather than practical tooling limitations.

VikSchema was designed around the realities of Figma.

The system adapts to the tool instead of fighting it.

From tool limitation to system architecture

Figma Constraints	VikSchema Approach
Limited Slot System	Slot-like structure implemented via properties
Limited Nested Injection	Controlled exposure at the right layer
Complex Instance Overrides	Reduced depth and override complexity
Lack of Deep Dynamic Composition	Predictable composition within tool boundries

This ensures:

- More Predictable Workflows
- Lower System Friction
- Improved Usability Across Teams
- Sustainable Long-Term Maintenance

Impact

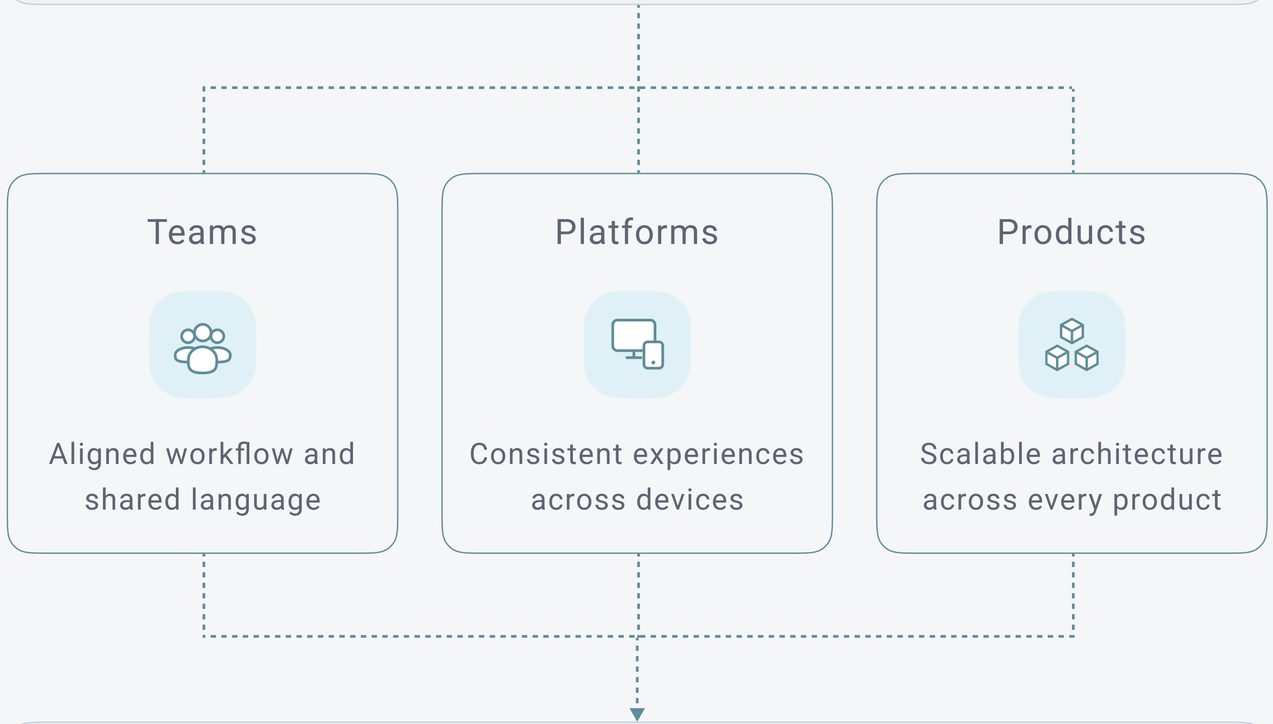
VikSchema transforms how teams build and scale products.

Instead of reacting to inconsistencies, teams build on a structure designed to prevent them.

The system creates:

- Reduced Cognitive Load
- Clearer Decision Boundaries
- Predictable Scaling Behavior
- Reusable Context-Aware Components
- Lower Maintenance Complexity
- Faster Product Evolution

The system that scales naturally across teams, platforms, and products.



Stronger systems, Faster teams, Better products.
Built on structure, not improvisation.

Reflection

A framework for scalable systems

The goal of VikSchema was not to create another collection of components.

It was to design a framework for building systems that scale.

The core shift was moving from:

Complexity-Based Organization → Responsibility-Based Architecture

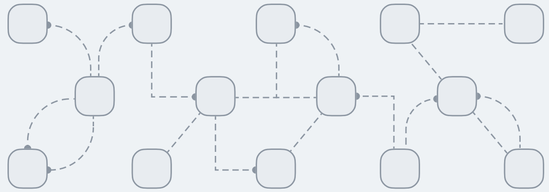
By separating structure, styling, layout, and context into independent layers, the system becomes easier to reason about, easier to maintain, and easier to scale over time.

Vikschema structure complexity through responsibility

Built for scale. Designed to last

Before

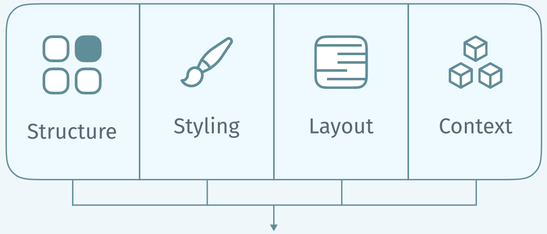
Complexity-Based Organization



Unclear boundaries. Overlapping responsibilities. High friction. Hard to scale.

After

Responsibility-Based Architecture



Clear boundaries. Defined responsibilities. Low friction. Built to scale.

VikSchema

VikSchema is not a collection of components.

Gene → Cell → Tissue → Organ → Organism → Colony

It is a system
for building systems.

Clear Responsibilities
No ambiguity.

Controlled Complexity
Scales with confidence.

Reusable Foundations
Built once. Used everywhere.

Sustainable Growth
Designed to evolve.